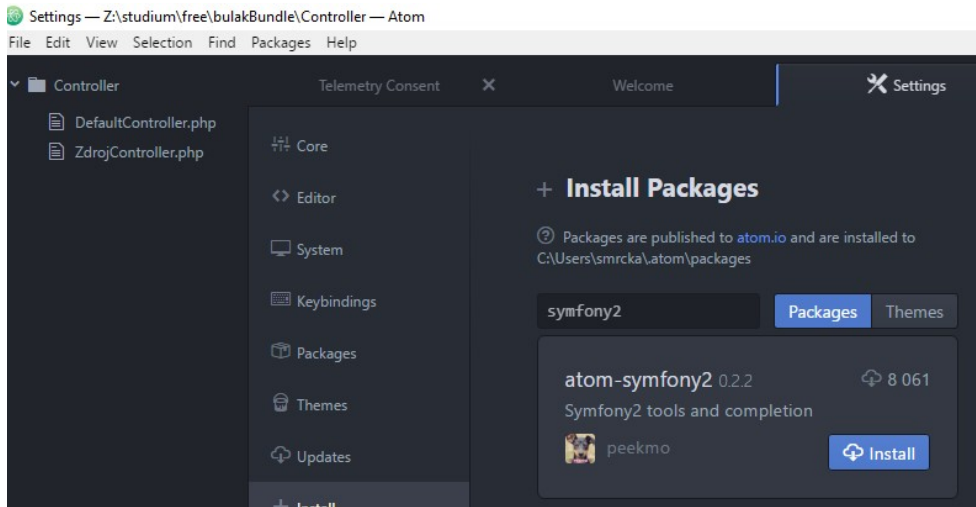


Obsah

1	Nová šablona jmena.html.twig.....	2
2	DUMP.....	4
2.1	Dump v HTML výstupu.....	6
3	Pole.....	6
4	Multidimenzionalni pole.....	10
5	If zda to existuje - pole.....	12
6	V praxi, pokud je výpis třída sudá lichá, měním vzhled.....	13
7	Vytvoření menu.....	16
8	Dokumentace ke twigu.....	19
9	Výpis aktuálního roku.....	20
10	CRUD.....	20
10.1.1	Cíl lekce	20
10.1.2	Teoretická část	20
10.1.3	-Praktické ukázky	21
11	Upozornění mazání.....	26
12	Rozdělení řádků formuláře a české pojmenování. Nezapomenout v use TextType.....	26
13	Vzhled formuláře.....	27
1	Pravidlo 5 10 20 pro kontroler.....	28
2	Některé věci dáme do pomocného souboru a pak volat use.....	28
3	PHP routování.....	30
4	Formuláře a entity.....	32
4.1	Přidáme tlačítko.....	37
5	Co se odesílá, tedy zpracování.....	38
6	Ve formuláři mohou být 2 tlačítka.....	40
7	Zobrazení odeslaných dat.....	42
8	Zobrazení chyb ve formuláři.....	46
9	Zabezpečení zadávání ve formuláři na úrovni serveru.....	48
10	Dokumentace Symfony validation.....	51
11	Callback – lze definovat vlastní funkci.....	53
12	Twig ovládání dalších vlastnosti funkce set.....	53

1 Nová šablona jmena.html.twig

Poznámka



Vytvoříme soubor jmena.html.twig

jmena.html.twig — C:\Users\smrcka\gopas — Atom

```
1 {% extends 'base.html.twig' %}
2
3 {% block body %}
4 FFFF
5 {{jmeno}}
6
7 {% endblock %}
8
```

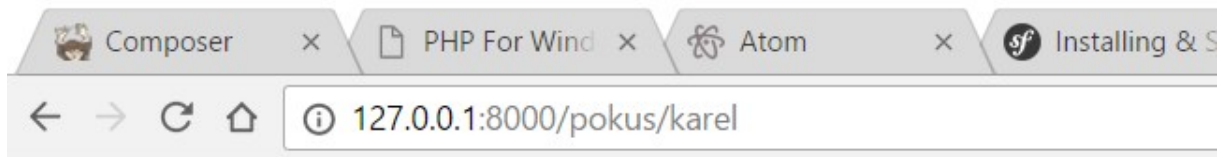
DefaultController.php — GopasBundle\DefaultController.php — C:\Users\smrcka\gopas — Atom

```
34 public function pozdravAction($name)
35 {
36     return new Response("Dobry den ".$name);
37 }
38
39 /**
40  * @Route("/session")
41  */
42 public function responseSessionAction(Request $request)
43 {
44
45     $session = $request->getSession();
46     $jmeno = $session->get("jmeno", "neznamy vojin");
47     return new Response("Dobry den, jmenujete se ".$jmeno);
48 }
49 /**
50  * @Route("/pokus/{jmeno}")
51  */
52 public function responseTwigAction($jmeno)
53 {
54
55     return $this->render('GopasBundle:Default:jmena.html.twig', array('jmeno'=>$jmeno));
56 }
57
58
59 }
```

Chci

V novém souboru se to spouští:

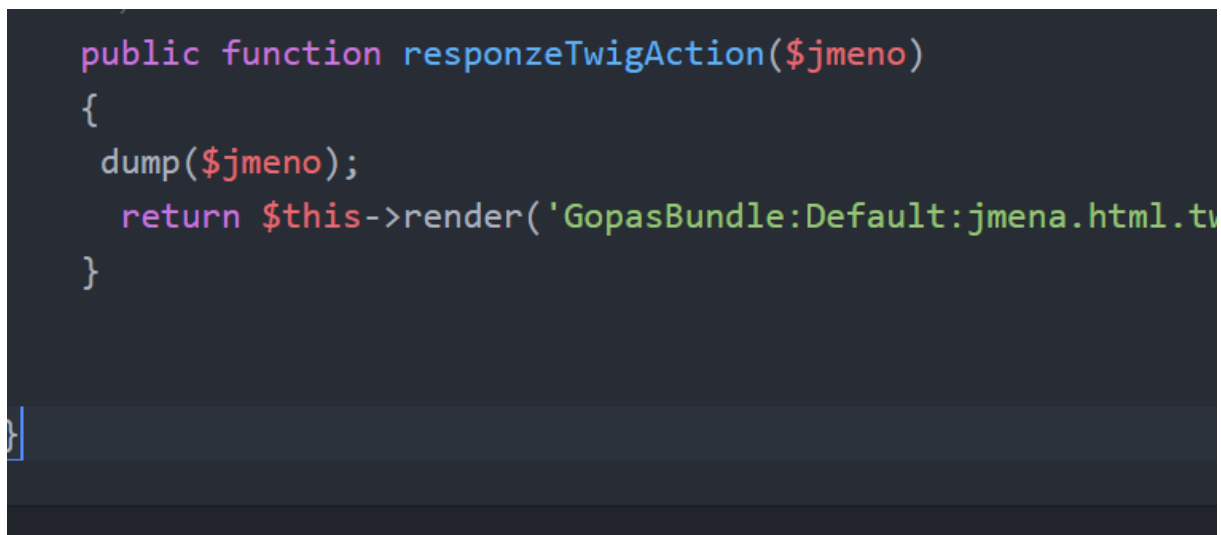
<http://localhost:8000/pokus2/jmeno>



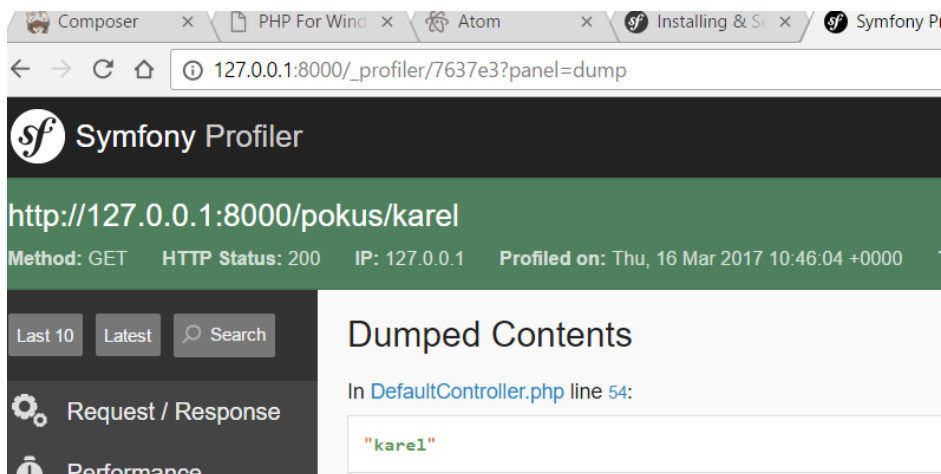
Nejaky nadpis, co je vsude

FFFFFF karel

2 DUMP



Nezapomenout pak v prohlížeči klepnout dole na dump



Přidáme `dump($_SERVER);`

```
public function responzeTwi
{
    dump($jmeno);
    dump($_SERVER);

    return $this->render('Gop
}
```

Výsledek v prohlížeči.

Dumped Contents

In `DefaultController.php` line 85:

```
"asax"
```

In `DefaultController.php` line 86:

```
array:22 [▼
  "DOCUMENT_ROOT" => "C:\franova_symfony1\projekt\web"
  "REMOTE_ADDR" => "127.0.0.1"
  "REMOTE_PORT" => "49434"
  "SERVER_SOFTWARE" => "PHP 7.1.3 Development Server"
  "SERVER_PROTOCOL" => "HTTP/1.1"
  "SERVER_NAME" => "127.0.0.1"
  "SERVER_PORT" => "8000"
  "REQUEST_URI" => "/pokus2/asax"
  "REQUEST_METHOD" => "GET"
  "SCRIPT_NAME" => "\app_dev.php"
  "SCRIPT_FILENAME" => "C:\franova_symfony1\projekt\web\app_dev.php"
  "PHP_SELF" => "\app_dev.php"
  "HTTP_HOST" => "127.0.0.1:8000"
  "HTTP_CONNECTION" => "keep-alive"
  "HTTP_UPGRADE_INSECURE_REQUESTS" => "1"
  "HTTP_USER_AGENT" => "Mozilla/5.0 (Windows NT 6.1) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/57.0.2987.133 Safari/537.36"
  "HTTP_ACCEPT" =>
"text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8"
  "HTTP_ACCEPT_ENCODING" => "gzip, deflate, sdch, br"
  "HTTP_ACCEPT_LANGUAGE" => "cs-CZ,cs;q=0.8"
  "HTTP_COOKIE" => "PHPSESSID=g1cjgsmemo7sud2ukubg0foma1"
  "REQUEST_TIME_FLOAT" => 1491691070.7955
  "REQUEST_TIME" => 1491691070
]
```

2.1 Dump v HTML výstupu

```
51
52  lic function responzeTwigAction($jmeno)
53  ~
54    mp($jmeno);
55  ~ mp($_SERVER);
56
57  return $this->render('GopasBundle:Default:jmena.html.twig',array('jmeno'=>$jmeno,'question' =>'Jak se mate?'));
58
59
60
```

```
{{ dump (slovo) }}
```

```
"ahoj"
```

```
{% dump (slovo) %}
```

200 @ uvod 339 ms 4.0 MB anon. 10 ms 2

Dumped Contents

In DefaultController.php line 64:

```
"ahoj"
```

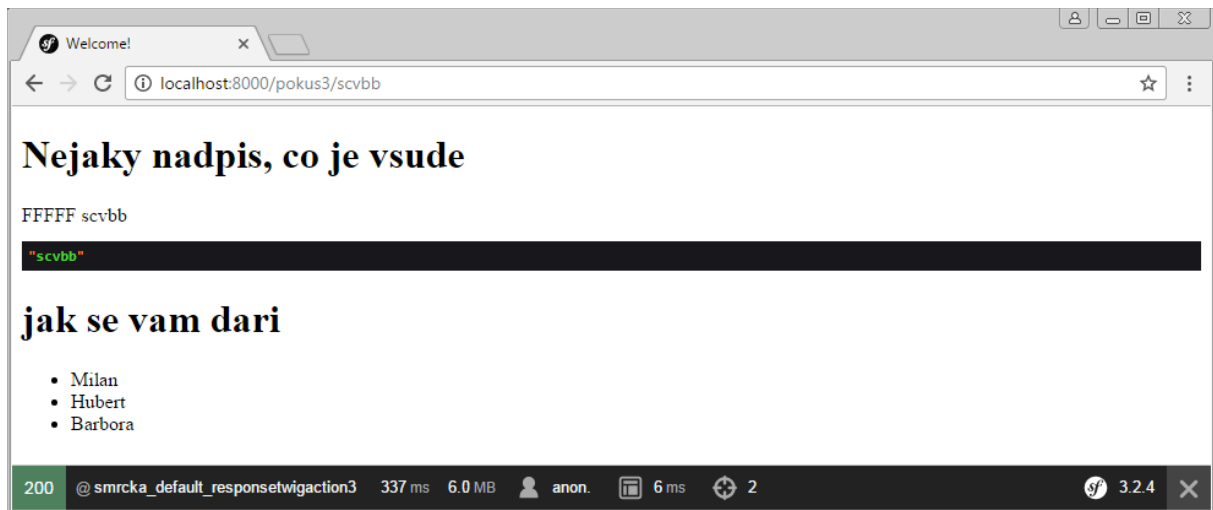
In default/uvod.html.twig line 12:

```
"ahoj"
```

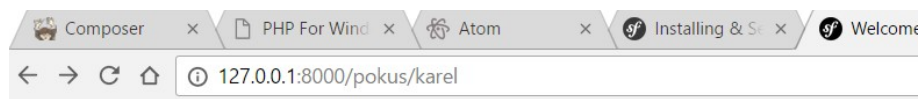
3 Pole

```
89      }
90
91
92
93  /**
94
95      * @Route("/pokus3/{jmeno}")
96      */
97      public function responseTwigAction3($jmeno)
98      {
99          dump($jmeno);
100      return $this->render('smrckaBundle:Default:jmena.html.twig',
101          array('jmeno'=>$jmeno,'question'=>'jak se vam dari',
102      | 'hosti'=>array('Milan','Hubert','Barbora'),));
103      }
104
```

```
1  {% extends 'base.html.twig' %}
2
3  {% block body %}
4  FFFFF
5  {{jmeno}}
6  {% dump(jmeno) %}
7  {{ dump(jmeno) }}
8  <h2>{{question}}</h2>
9  <ul>
10  {% for host in hosti %}
11  <li>{{host}}</li>
12
13
14  {% endfor %}
15  </ul>
16
17
18  {% endblock %}
19 |
```



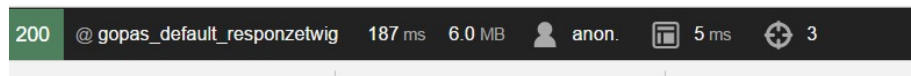
```
config.yml  AppBundle.php  GopasBundle.php  DefaultController.php
1  {% extends 'base.html.twig' %}
2
3  {% block body %}
4  FFFFF
5  {{jmeno}}
6  {% dump(jmeno) %}
7  {{ dump(jmeno) }}
8  {% endblock %}
9
```

Nejaky nadpis, co je vsude

FFFFF karel

"karel"

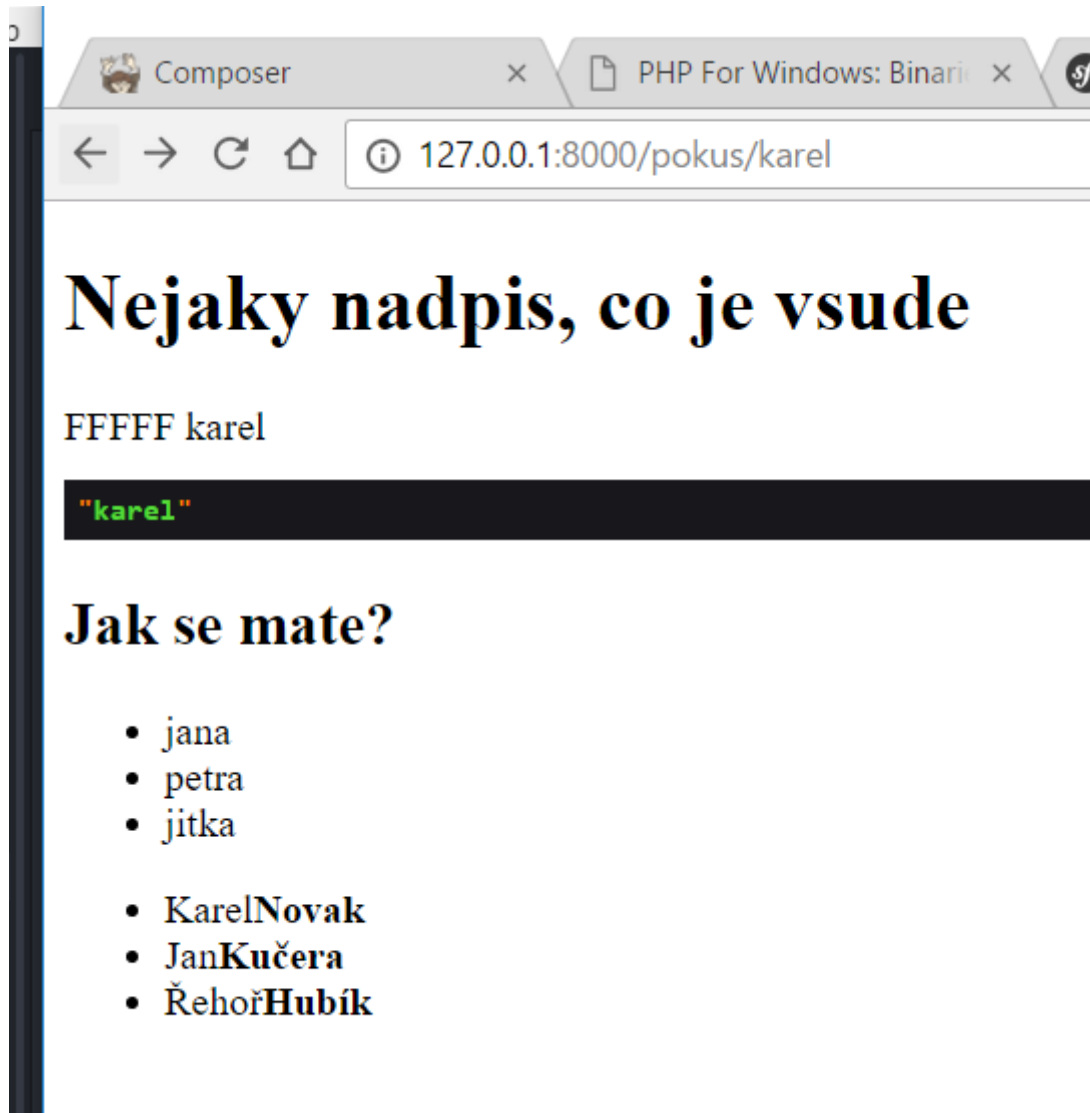


4 Multidimenzionalni pole

```
config.yml AppBundle.php GopasBundle.php DefaultController.php jmena.html.twig config
1 {% extends 'base.html.twig' %}
2
3 {% block body %}
4 FFFF
5 {{jmeno}}
6 {% dump(jmeno) %}
7 {{ dump(jmeno) }}
8 <h2>{{question}}</h2>
9 <ul>
10 {% for host in hosti %}
11 <li>{{host}}</li>
12
13
14 {% endfor %}
15 </ul>
16 <ul>
17 <li>jmeno <strong>Prijmeni</strong></li>
18 <li>jmeno <strong>Prijmeni</strong></li>
19 <li>jmeno <strong>Prijmeni</strong></li>
20
21 </ul>
22
23 {% endblock %}
```

```
public function responseTwigAction($jmeno)
{
    dump($jmeno);
    dump($_SERVER);

    return $this->render('GopasBundle:Default:jmena.html.twig',array('jmeno'=>$jmeno,
        'question' =>'Jak se mate?',
        'hosti'=>array('jana','petra','jitka'),
        'studenti'=>array(
            array('jmeno'=>'Karel', 'prijmeni'=>'Novak'),
            array('jmeno'=>'Jan', 'prijmeni'=>'Kučera'),
            array('jmeno'=>'Řehoř', 'prijmeni'=>'Hubík'),
        )
    ));
}
```



```

config.yml  AppBundle.php  GopasBundle.php  DefaultController.php  jmena.html.twig  config.cson  index.h
1  {% extends 'base.html.twig' %}
2
3  {% block body %}
4  FFFFF
5  {{jmeno}}
6  {% dump(jmeno) %}
7  {{ dump(jmeno) }}
8  <h2>{{question}}</h2>
9  <ul>
10  {% for host in hosti %}
11  <li>{{host}}</li>
12
13
14  {% endfor %}
15  </ul>
16  <ul>
17  {% for student in studenti %}
18  <li>{{student.jmeno}}<strong>{{student.prijmeni}}</strong></li>
19
20  {% endfor %}
21  </ul>
22
23  {% endblock %}
24

```

5 If zda to existuje - pole

```

/**
 * @Route("/pokus/{jmeno}")
 */
public function responseTwigAction($jmeno)
{
    dump($jmeno);
    dump($_SERVER);

    return $this->render('GopasBundle:Default:jmena.html.twig', array('jmeno'=>$jmeno,
        'question' =>'Jak se mate?',
        // 'hosti'=>array('jana', 'petra', 'jitka'),
        'studenti'=>array(
            array('jmeno'=>'Karel', 'prijmeni'=>'Novak'),
            array('jmeno'=>'Jan', 'prijmeni'=>'Kučera'),
            array('jmeno'=>'Řehoř', 'prijmeni'=>'Hubík'),
        ),
    ));
}

```

```
{% if hosti is defined %}
<ul>
  {% for host in hosti %}
  <li>{{host}}</li>

  {% endfor %}
</ul>
{% endif %}
```

Už se neobjeví chyba. Ověří se, zda je proměnná `hosti` naplněná.

```
{% if hosti is defined and hosti %}
```

```
{
  dump($jmeno);
  dump($_SERVER);

  return $this->render('GopasBundle:Default:jmena.html.twig', array('jmeno'=>$jmeno,
    'question' =>'Jak se mate?',
    'hosti'=>array(),
    'studenti'=>array(
      array('jmeno'=>'Karel', 'prijmeni'=>'Novak'),
      array('jmeno'=>'Jan', 'prijmeni'=>'Kučera'),
      array('jmeno'=>'Řehoř', 'prijmeni'=>'Hubík'),
    ),
  ));
}
```

6 V praxi, pokud je výpis třída sudá lichá, měním vzhled

Css cycle

```
<ul>
  {% for student in studenti %}
  <li class="{ cycle(['suda', 'licha'], loop.index) }">{{student.jmeno}}<strong>{{student.prijmeni}}</strong></li>

  {% endfor %}
```

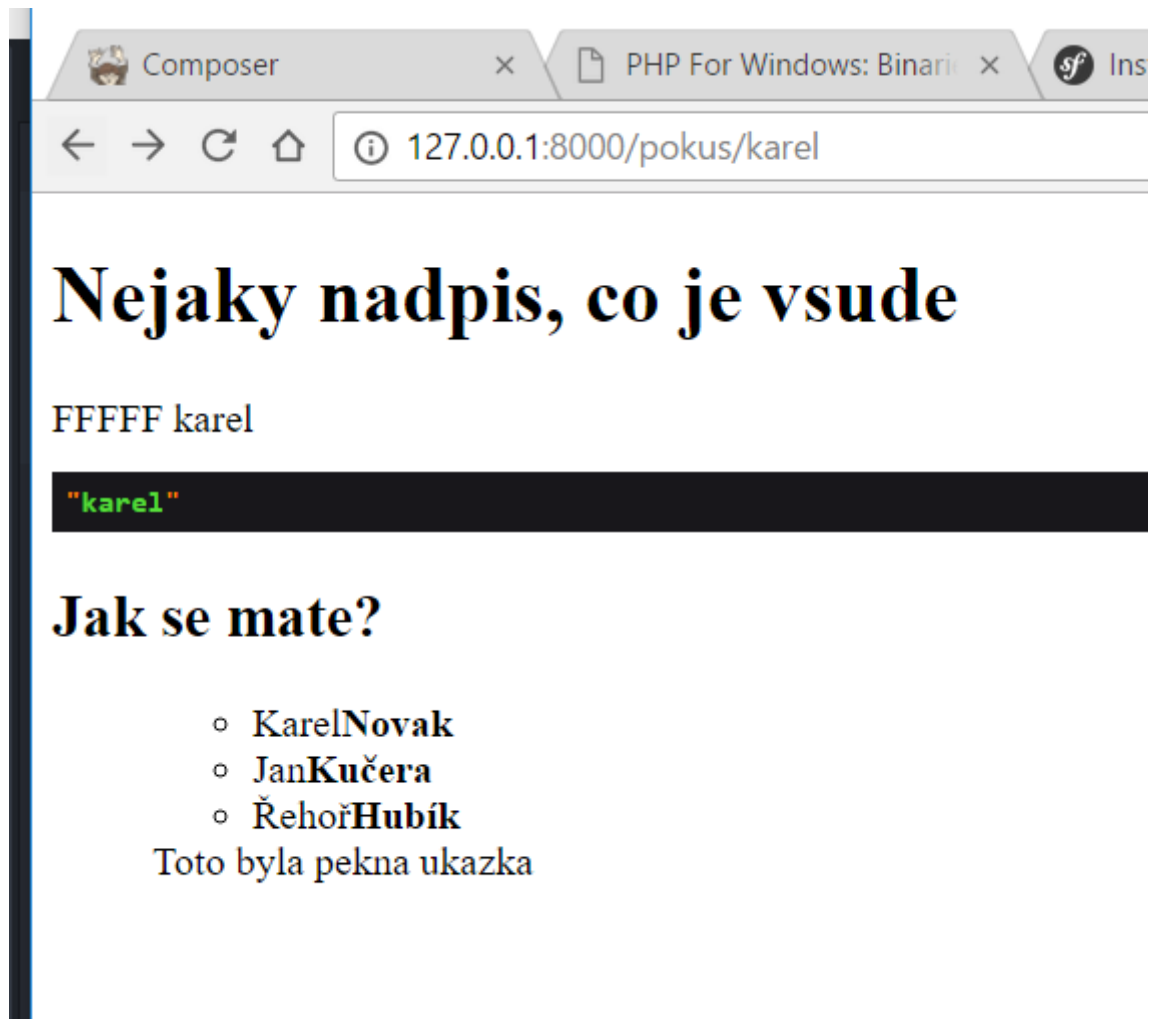
Pak jen definovat třídu licha suda.

Vložení nového bloku do twig default.

```
        {% block body %}
<p>Je tu nejaky text</p>
        {% endblock %}
    {% block footer %}{% endblock %}

    {% block javascripts %}{% endblock %}
</body>
```

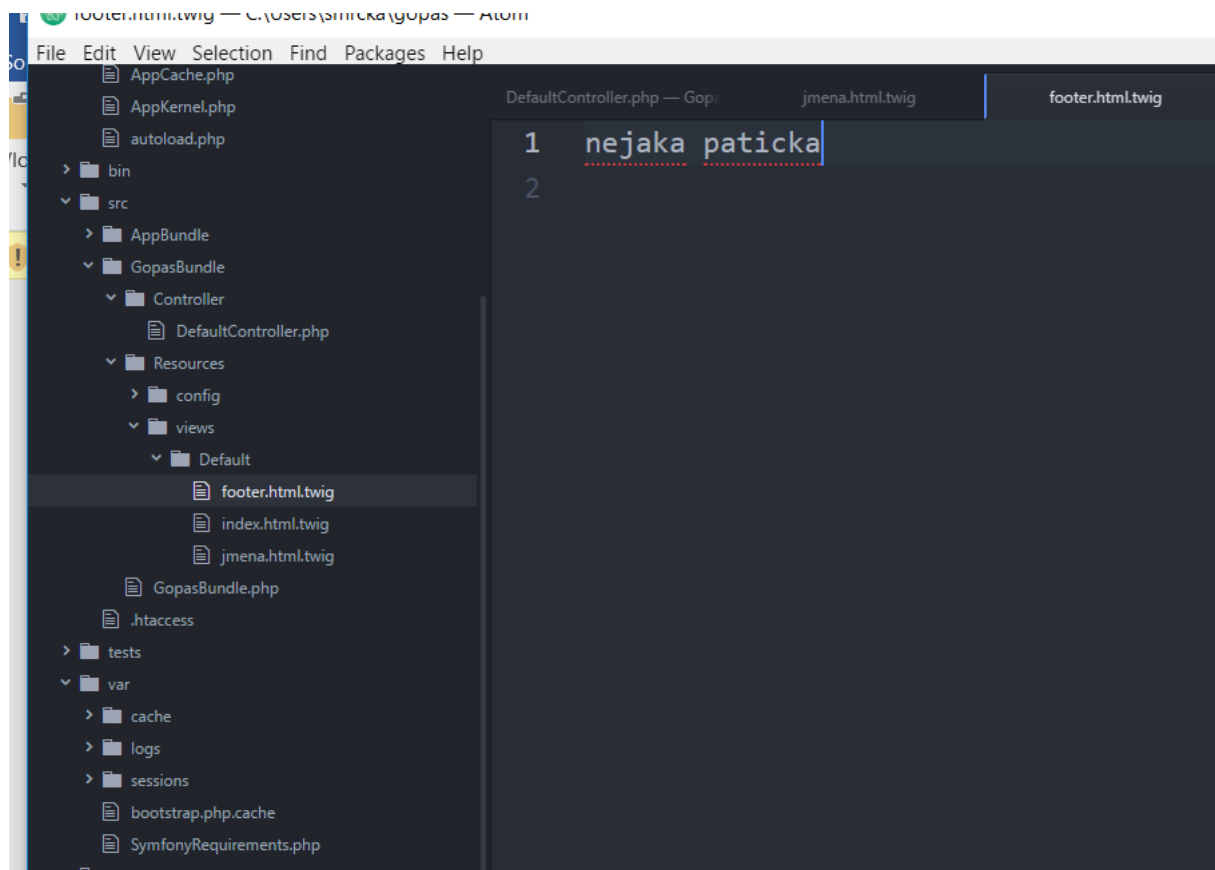
```
DefaultController.php — Gopasli | jmena.html.twig | config.cson | index.html.twig
6  {% dump(jmeno) %}
7  {{ dump(jmeno) }}
8  <h2>{{question}}</h2>
9
10 {% if hosti is defined and hosti %}
11 <ul>
12     {% for host in hosti %}
13     <li>{{host}}</li>
14
15
16 {% endfor %}
17 </ul>
18 {% endif %}
19 <ul>
20     {%# komentar %}
21     <ul>
22         {% for student in studenti %}
23         <li class="{ cycle(['suda', 'licha'], loop.index) }">{{student.jme
24
25     {% endfor %}
26     </ul>
27
28     {% endblock %}
29
30     Toto byla pekna ukazka
31
```



```
</ul>  
{% for pracovnik in pracovnici %}  
  <li class="{{ cycle(['suda','licha'],loop.index) }}" >{{ pracovnik.jmeno }} {{ pracovnik.prijmeni }}</li>  
{% endfor %}  
</ul>
```

Vytvořit nový soubor

footer.html.twig



7 Vytvoření menu

příklad sablony

soubor: hlavni.html.twig

```
<!DOCTYPE html>
<html>
<head>
    {% block head %}
        <meta charset="UTF-8" />
        <title>{% block title %}Vítej!{% endblock %}</title>
        {% block stylesheets %}
            <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css"
">
        {% endblock %}
        {% block javascripts %}

    {% endblock %}
    {% endblock %}
</head>
<body>
<div class="row">
    <div class="col-sm-12 hlavicka">
        <h1 class="hlavni-nadpis">Hlavní strana</h1>
    </div>
</div>
<div class="row">
    <nav class="navbar navbar-default">
        <div class="container-fluid">
```



```

        <ul class="nav navbar-nav">
            <li><a href="{{ path('uvod') }}">Úvod</a></li>
            <li><a href="{{ path('prvni') }}">První</a></li>
        </ul>
    </div>
</nav>
</div>

{% block content %}{% endblock %}
{% block footer %}{% endblock %}
{% block sidebar %}
    dedeno sitebar
{% endblock %}

</body>
</html>

```

uvod.html.twig

```

{% extends 'hlavni.html.twig' %}

{% block title %} {% endblock %}
{% block head %}
    {{ parent() }}
{% endblock %}
{% block content %}
    Toto je úvodní stránka, která dědí od hlavního layoutu.

    Poslany text {{ slovo }}
{% endblock %}

```

prvni.html.twig

```

{% extends 'hlavni.html.twig' %}

{% block title %} {% endblock %}
{% block head %}
    {{ parent() }}
{% endblock %}
{% block content %}
    Toto je podstránka, která dědí od hlavního layoutu.
{% endblock %}
{% block sidebar %}
    <h3>Table Of Contents</h3>
{% endblock %}

```

Výsledek

Hlavní strana

Úvod První

Toto je úvodní stránka, která dědí od hlavního layoutu. Poslany text ahoj dedeno sidebar

V případě, že použiji v souboru první.html.twig

```
{% block sidebar %}
    <h3>Table Of Contents</h3>
    {{ parent() }}
{% endblock %}
```

Místo

Hlavní strana

Úvod První

Toto je podstránka, která dědí od hlavního layoutu.

Table Of Contents

se objeví

Hlavní strana

Úvod První

Toto je podstránka, která dědí od hlavního layoutu.

Table Of Contents

dedeno sitebar

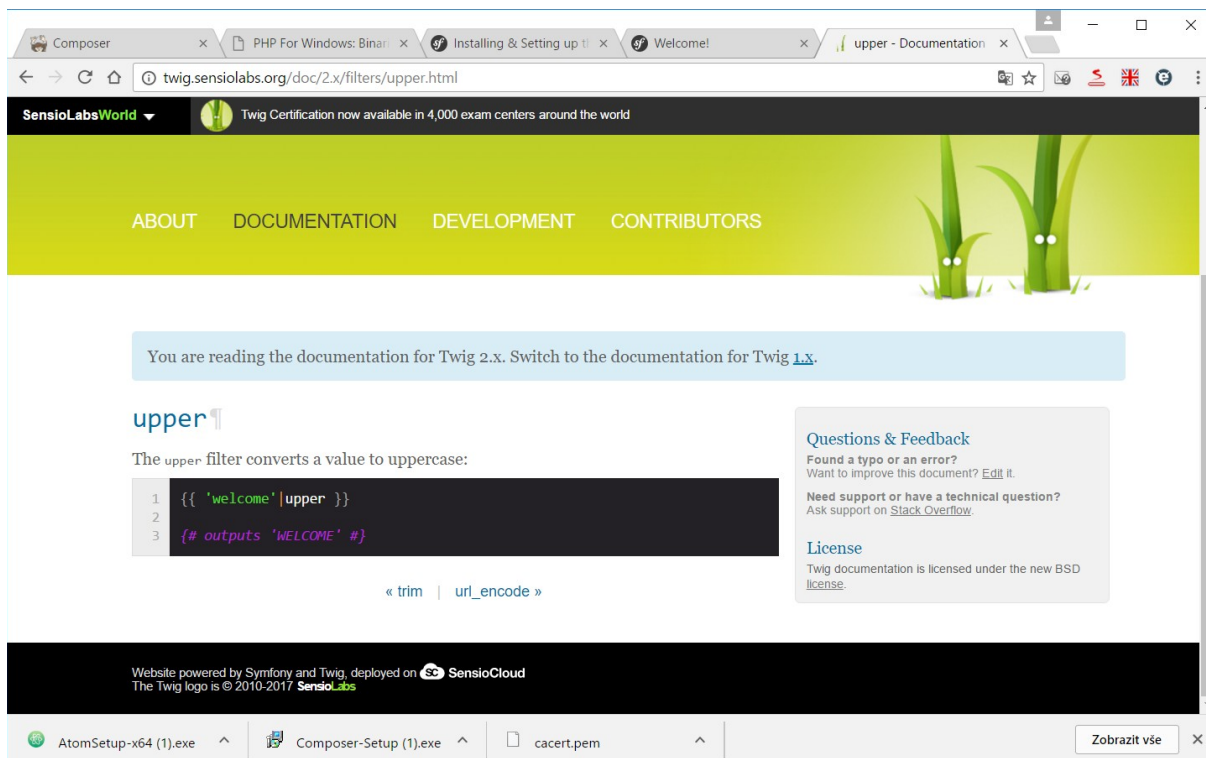
8 Dokumentace ke twigu

<https://twig.symfony.com/>

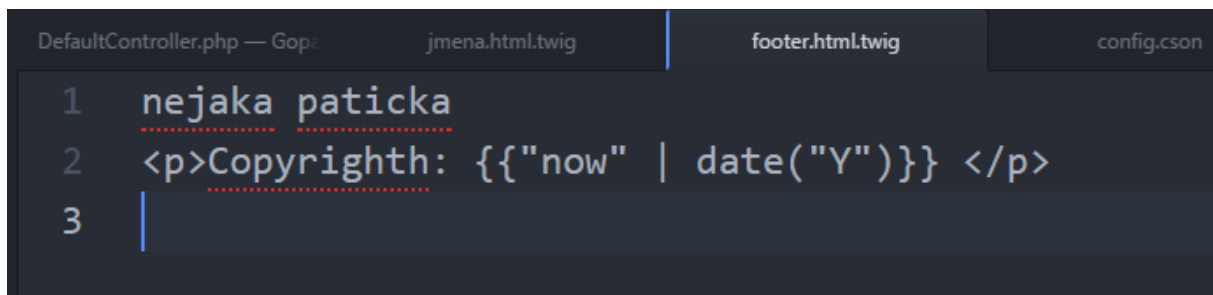
Filtr

Vezme například jmeno, může něco odfiltrovat atd

Upper



9 Výpis aktuálního roku

A screenshot of a code editor with a dark theme. The editor has four tabs at the top: 'DefaultController.php — Gop...', 'jmena.html.twig', 'footer.html.twig', and 'config.cson'. The 'jmena.html.twig' tab is active. The code in the editor is as follows:

```
1 nejaka paticka
2 <p>Copyright: {{ "now" | date("Y") }} </p>
3
```

Filtr se dá řetězit

```
{{jmeno | upper | ...}}
```

10 CRUD

10.1.1 Cíl lekce

Cílem této lekce je seznámení se s Doctrine a vytvoření vlastní entity a rozhraní CRUD pomocí Doctrine.

10.1.2 Teoretická část

10.1.2.1 Doctrine

Doctrine je PHP knihovna pro práci s databází. Doctrine je tzv. ORM (object relational mapping) což znamená, že pomocí ní můžeme mapovat objekty na relační databázi (Doctrine umí i naopak). Toto znamená, že v podstatě nemusíme řešit, že máme nějakou databázi, ale pracujeme pouze s objekty. Doctrine má pro dotazy dokonce svůj vlastní jazyk DQL (Doctrine Query Language). Tento jazyk je velice podobný jazyku SQL (Select Query Language). Objekty se kterými Doctrine pracuje se nazývají entity a ty nesou data. Tato knihovna navíc použité dotazy optimalizuje a uchovává pro další použití. Tímto postupem může naši aplikaci i vcelku zrychlit.

ORM

ORM je programovací technika, která zajišťuje automatickou konverzi dat mezi relační databází a objektově orientovaným programovacím jazykem. Tato technika, v některých případech, úplně odstíní programátora od znalosti jazyka SQL. Použití této techniky usnadní zejména provádění běžných databázových operací jako mazání, editování či přidávání dat. Důležitou funkcí ORM je trvalé uchování dat tzn. aby data aplikace, která jsou v operační paměti, se během nečekaného chování aplikace neztratila. Nejdůležitější funkcí ORM je synchronizace mezi objekty, které jsou používány v aplikaci a databázovým systémem, kde jsou uloženy.

Více info o ORM [zde](#).

10.1.2.2 *CRUD*

CRUD je generované rozhraní pro práci s entitou. Jak již název prozrazuje (create, read, update, delete) zahrnuje základní operace nad entitou. Tyto operace jsou vytvoření, zobrazení, změna, a odstranění záznamu v entitě (tabulce). CRUD vygeneruje controller v námi zadaném bundlu nad námi zadanou entitou a také čtyři šablony (pro každou operaci jednu).

10.1.3 **-Praktické ukázky**

Jako první věc si ukážeme jak konfigurovat databázi a jak ji vytvořit. Nyní si ukážeme soubory ve kterých se databáze konfiguruje. Jsou to dva soubory a najdeme je ve složce `app->config`. Jsou to soubory `config.yml` a `parameters.yml`. Jestliže vyvíjíme projekt lokálně a používáme databázi **MySQL** není třeba měnit jakékoli nastavení (pokud nastane problém s ovladačem je nutné ho povolit v souboru `php.ini`, který jsme si vytvořili při instalaci). Jestliže chceme používat databázi **SQLite** (jednosouborová databáze) je nutné provést tyto změny. V souboru `config.yml` u konfigurace Doctrine změníme ovladač na `pdo_sqlite`. Druhá změna v tomto souboru je ta, že přidáme parametr `path`.

```
# Doctrine Configuration
doctrine:
  dbal:
    driver: pdo_sqlite
    host: '%database_host%'
    port: '%database_port%'
    dbname: '%database_name%'
    user: '%database_user%'
    password: '%database_password%'
    path: '%database_path%'
    charset: UTF8
```

Jelikož jsme přidali nový parametr musíme upravit i soubor *parameters.yml*. Ještě než ho upravíme, vytvoříme si ve složce *var* složku *data* ve které bude databáze uložena. V souboru *parameters.yml* přidáme parametr *path* takto. Zde na pořadí parametrů nezáleží, pouze musejí být správně odsazené.

```
parameters:
  database_path: "%kernel.root_dir%../var/data/data.sqlite"
  database_host: 127.0.0.1
  database_port: null
  database_name: symfony
  database_user: root
  database_password: null
  mailer_transport: smtp
  mailer_host: 127.0.0.1
  mailer_user: null
  mailer_password: null
  secret: c4704a4173c8251e5422eee7eece6ea3b7c41592
```

10.1.3.1 Ukázka kódu pro parametr path

Pokud však budeme vyvíjet na nějakém serveru je třeba v souboru *parameters.yml* nastavit odpovídající hodnoty. Zde je ukázka připojení ke školní databázi, jestliže je projekt nahráný na školním serveru. Toto připojení se nastaví pouze v souboru *parameters.yml*.

```
parameters:
  database_host: localhost
  database_port: null
  database_name: jmeno_databaze # většinou váš login
  database_user: jmeno_uzivatele # většinou váš login
  database_password: heslo # heslo, které jste si nastavili
```

A teď se již vrhneme na tvorbu databáze a entit. Databáze se vytvoří jednoduchým příkazem.

php bin/console doctrine:database:create

```
C:\projekty\vyukovy_web>php bin/console doctrine:database:create
```

Jestliže máme databázi vytvořenou je čas vytvořit si naši entitu. My si vytvoříme entitu s názvem skladiste. Bude mít tři sloupce:

- Nazev - string (255)
- Cena - float
- Dostupnost - boolean

Skladiste
Nazev
Cena
Dostupnost

Na vytvoření entity opět použijeme Doctrine, která naši práci s databází zjednoduší. Použijeme tento příkaz.

php bin/console doctrine:generate:entity

Nejprve musíme zadat, jméno entity a jméno bundlu ve které ji chceme vygenerovat. My zvolíme defaultní bundle (AppBundle) a název entity skladiste. Jako druhá věc se musí zadat konfigurační formát. Zde necháme annotation, což je defaultní možnost.

```
The Entity shortcut name: AppBundle:skladiste
```

```
Determine the format to use for the mapping information.
```

```
Configuration format (yaml, xml, php, or annotation) [annotation]:
```

Nyní již následuje samotná tvorba entity. Zde si definujeme jaké sloupce bude entita obsahovat. První je zde název sloupce. Následuje typ sloupce, další volba je zda-li může být nulový. Jako poslední je zde určení jedinečnosti v rámci entity. Sloupec *id* se do naší entity přidá defaultně sám.

```
New field name (press <return> to stop adding fields): Cena
Field type [string]: float
Is nullable [false]:
Unique [false]:
```

Po vytvoření naší entity je nutné zadat tento příkaz.

php bin/console doctrine:schema:update --force

Proto, aby se změna projevila v databázi. Po tomto kroku se v námi zadaném bundle (v našem případě AppBundle) ve složce Entity vytvořil .php soubor s názvem, který jsme zadali při vytváření (skladiste). Zde se nachází naše vygenerovaná entita. Nyní si ukážeme jak vygenerovat jednoduchou správu této entity. K tomu nám poslouží CRUD. Opět si toto vygenerujeme pomocí Doctrine příkazem.

php bin/console generate:doctrine:crud

Po potvrzení tohoto příkazu musíme zadat jméno a umístění entity pro kterou chceme CRUD generovat. V našem případě AppBundle:skladiste. Jako druhé musíme vybrat jestli chceme vygenerovat i "psané" akce, což je mazání, vytvoření, upravení. Zde je defaultní možnost "no" a tuto možnost musíme přepsat na "yes".

```
The Entity shortcut name: AppBundle:skladiste

By default, the generator creates two actions: list an
You can also ask it to generate "write" actions: new,

Do you want to generate the "write" actions [no]? yes
```

Dále musíme opět vybrat konfigurační formát, kde vybereme annotation (defaultní možnost). Poté vybereme jaký prefix budeme chtít pro toto rozhraní používat. Doporučuji nechat defaultní, ale pokud chceme můžeme si zadat cokoli. Jako poslední je jenom potvrzení této generace.


```
Configuration format (yaml, xml, php, or annotation) [annotation]:
```

```
Determine the routes prefix (all the routes will be "mounted" under this  
prefix: /prefix/, /prefix/new, ...).
```

```
Routes prefix [/skladiste]:
```

Po vygenerování máme kompletní správu naší entity, která ale není graficky nejlepší. Ještě musíme zadat příkaz pro update databáze.

php bin/console doctrine:schema:update --force

Tento vzhled můžeme samozřejmě upravit podle sebe. Můžeme ho měnit v šablonách, které se nám vygenerovali. Tyto šablony jsou ve složce *app->Resources->views->složka s naším jménem, které jsme zadali*. Zde se nacházejí čtyři šablony, každá pro jinou akci. Tyto šablony my můžeme měnit tak jak budeme potřebovat. Můžeme zde dělit do divu nebo přidat třídy Bootstrapu, jestliže je máme nalinkované.

Když naši CRUD aplikaci poprvé spustíme s routou (prefixem), kterou jsme si vybrali uvidíme toto.



Skladistes list

IdNázevCenaDostupnostActions

- [Create a new skladiste](#)

Tady si ukážeme menší ukázkou jak lze editovat například formulář pro editaci dat (*app->Resources->views->skladiste->edit.html.twig*), aby vypadal trochu lépe. Uděláme to tak že pouze nahradíme tento řádek kódem, který je níže.

```

{{ form_start(edit_form) }}
    {{ form_widget(edit_form) }}
    <input type="submit" value="Edit" />
{{ form_end(edit_form) }}

```

10.1.3.2 Ukázka kódu pro úpravu šablony

V prohlížeči vypadá tato úprava následovně.

← → ↻ 🏠 localhost:8000/skladiste/1/edit

Skladiste edit

Název

Cena

Dostupnost ☐

- [Back to the list](#)
-

Nakonec si ještě ukážeme jak lze napojit formulář (lekce 6) na entitu. A jak do této entity následně uložit data zadaná ve formuláři. Aby vše fungovalo správně musí být entita vytvořena pomocí Doctrine. Při tvorbě formuláře budeme postupovat jako v šesté lekci. Změna bude v tom, že po kontrole jestli byla data odeslána tyto data zapíšeme do databáze. To se provede takto.

```

if ($form->isSubmitted() && $form->isValid()) {
    $task = $form->getData();
    $em = $this->getDoctrine()->getManager();
    $em->persist($task);
    $em->flush();
}

```

11 Upozornění mazání

```
<a href="{{ path('firma_delete', {'firma': firma.id}) }}" class="btn btn-danger"
onclick="return confirm('Opravdu si přejete smazat vybranou firmu?')" title="Odstranit firmu">
```

Případně

```
<input type="submit" value="Delete" onclick="return confirm('Opravdu si přejete smazat vybranou firmu?')" title="Odstranit firmu">
```

12 Rozdělení řádků formuláře a české pojmenování. Nezapomenout v use TextType

```
<?php
```

```
namespace AppBundle\Form;
```

```
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\Extension\Core\Type\TextType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;
```

```
class StrojType extends AbstractType
{
    /**
     * {@inheritdoc}
     */
    public function buildForm(FormBuilderInterface $builder, array $options)
    {
        $builder->add('nazev', TextType::class, [
            'label' => 'Název',
            'required' => true,
        ]);
        $builder->add('cena', TextType::class, [
            'label' => 'Cena',
            'required' => true,
        ]);
    }
}
```

```
/**
 * {@inheritdoc}
 */
public function configureOptions(OptionsResolver $resolver)
{
    $resolver->setDefaults(array(
```

```

        'data_class' => 'AppBundle\Entity\Stroj'
    ));
}

/**
 * {@inheritdoc}
 */
public function getBlockPrefix()
{
    return 'appbundle_stroj';
}

}

```

13 Vzhled formuláře

```

{% extends 'base.html.twig' %}
{% block stylesheets %}
<link rel="stylesheet"
href="{{ asset
('https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css') }}"
">
    {# {% form_theme form 'bootstrap_4_layout.html.twig' %} #}

    {% form_theme form 'bootstrap_4_horizontal_layout.html.twig' %}
{% endblock %}
{% block body %}
<div class="container">
    <h1>Stroj creation</h1>

    {{ form_start(form) }}
    <div style="width: 800px">
        {{ form_widget(form) }}
    </div>
    <input type="submit" value="Create" />
    {{ form_end(form) }}

    <ul>
        <li>
            <a href="{{ path('stroj_index') }}">Back to the list</a>
        </li>
    </ul>
</div>
{% endblock %}

```

Stroj creation

Název

Cena

Create

- [Back to the list](#)

1 Pravidlo 5 10 20 pro kontroler

Definováno 5 nebo méně proměnných

10 kolem 10 akcí

20 – v každé akci by nemělo být více než řádků

2 Některé věci dáme do pomocného souboru a pak volat use

V rámci bunde nova složka utils

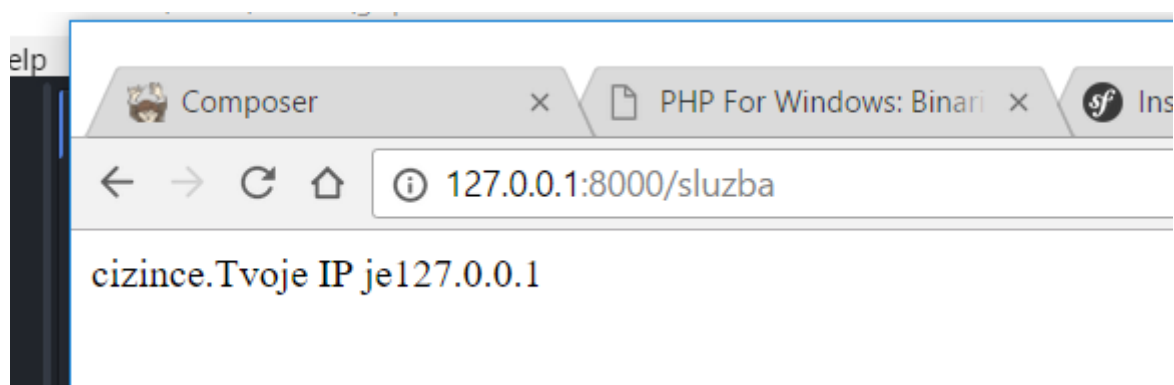
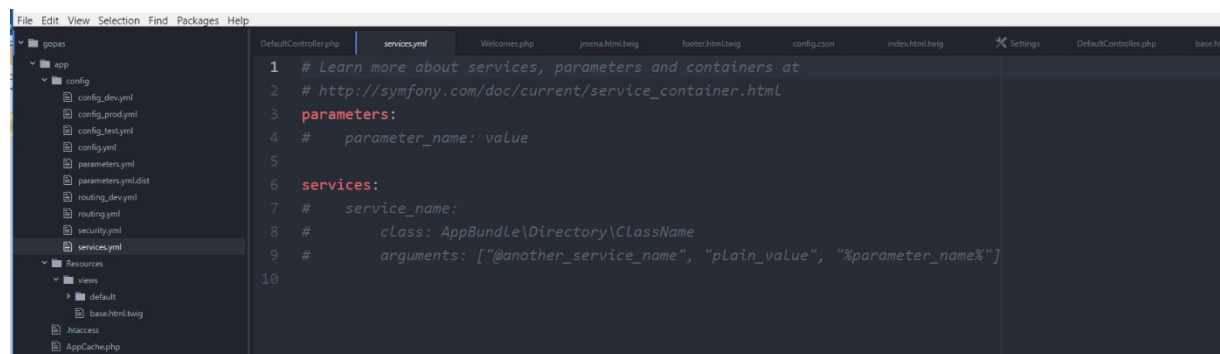
V ní soubor Welcome.php

Tedy dám to do souboru Welcome.php

```
use Sensio\Bundle\FrameworkExtraBundle\Configuration\Route;

use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\HttpFoundation\Request;
use GopasBundte\Utils\Welcomer;

class DefaultController extends Controller
{
    /**
```



```

    array('jmeno'=>'Jan', 'prijmeni'=>'Kucera'),
    array('jmeno'=>'Řehoř', 'prijmeni'=>'Hubík'),
    ),
    ));

}

/**
 * @Route("/sluzba", name="ukazkaservices" )
 */
public function responseServicesAction(Request $request)
{
    $welcomer = $this->get('app.welcomer'); //kde ji vezem, v appconfig servicess.yml
    $message = $welcomer->create("cizince");
    return new Response($message);
}
}

```

```

DefaultController.php | services.yml | Welcomer.php | jmena.html.twig | footer.html.twig | config.cson | index.html.twig | Settings | DefaultC
1 # Learn more about services, parameters and containers at
2 # http://symfony.com/doc/current/service_container.html
3 parameters:
4 #     parameter_name: value
5
6 services:
7 #     service_name:
8 #         class: AppBundle\Directory\ClassName
9 #         arguments: ["@another_service_name", "plain_value", "%parameter_name%"]
10 app.welcomer:
11     class: GopasBundle\utils\Welcomer
12     arguments: [cau]
13

```

Welcomer.php — C:\Users\smrcka\gopas — Atom

File Edit View Selection Find Packages Help

```

1 <?php
2 namespace GopasBundle\utils;
3 class Welcomer
4 {
5     private $intro;
6
7     public function __constructor($text)
8     {
9         $this->intro = $text;
10    }
11    //volam funkci intro, vrati to fci intro
12    public function create($name)
13    {
14        return $this->intro.' '.$name.' .Tvoje IP je'. $_SERVER['REMOTE_ADDR'];
15    }
16
17 }
18
19
20 }
21

```

gopas

- app
 - config
 - config_dev.yml
 - config_prod.yml
 - config_test.yml
 - config.yml
 - parameters.yml
 - parameters.yml.dist
 - routing_dev.yml
 - routing.yml
 - security.yml
 - services.yml
 - Resources
 - views
 - default
 - base.html.twig
 - .htaccess
 - AppCache.php
 - AppKernel.php
 - autoload.php
 - bin
 - console
 - symfony_requirements
 - src
 - AppBundle
 - GopasBundle
 - Controller
 - DefaultController.php
 - Resources
 - config
 - views

3 PHP routování

1. anotace

```
/**
 * @Route("/sluzba",name="ukazkaservices")
 */
```

Jinak je možné použít z dokumentace routování

1. Yml
- 2.

<http://symfony.com/doc/current/routing.html>

Anotace nesrozumitelnější.

```
Annotations  YAML  XML  PHP

1  // src/AppBundle/Controller/BlogController.php
2  namespace AppBundle\Controller;
3
4  use Symfony\Bundle\FrameworkBundle\Controller\Controller;
5  use Sensio\Bundle\FrameworkExtraBundle\Configuration\Route;
6
7  class BlogController extends Controller
8  {
9      /**
10       * Matches /blog exactly
11       *
12       * @Route("/blog", name="blog_list")
13       */
14       public function listAction()
15       {
16           // ...
17       }
18
19       /**
20       * Matches /blog/*
21       *
22       * @Route("/blog/{slug}", name="blog_show")
23       */
24       public function showAction($slug)
25       {
26           // $slug will equal the dynamic part of the URL
27           // e.g. at /blog/yay-routing, then $slug='yay-routing'
28
29           // ...
30       }
31  }
```

4 Formuláře a entity

Entity v symfony slouží jako objekt, jehož data získáme přes formulář.

Definujeme entitu pro ukolníček a přes formulář budeme naplňovat entitu.

Vyrobíme v GopasBundle složku **Entity** a v ní soubor Task.php

```
DefaultController  .htaccess  formular.html.twig  Task.php  services.yml  Welcomer.php  jmena.html.twig  footer

1  <?php
2  namespace GopasBundle\Entity;
3
4  class Task{
5  //promenna task
6  //vsechny protected, vracet gettr nap pro task getTask
7  // opak gettr je _set_oid_output_format
8
9  //mame entitu s jednou vlastnosti
10     protected $task;
11
12     public function getTask(){
13
14
15         return $this->task;
16
17     }
18
19     public function setTask($task){
20
21         $this->task=$task;
22     }
23 }
24
```

Dále se v Controlleru vytvoří routa formular


```

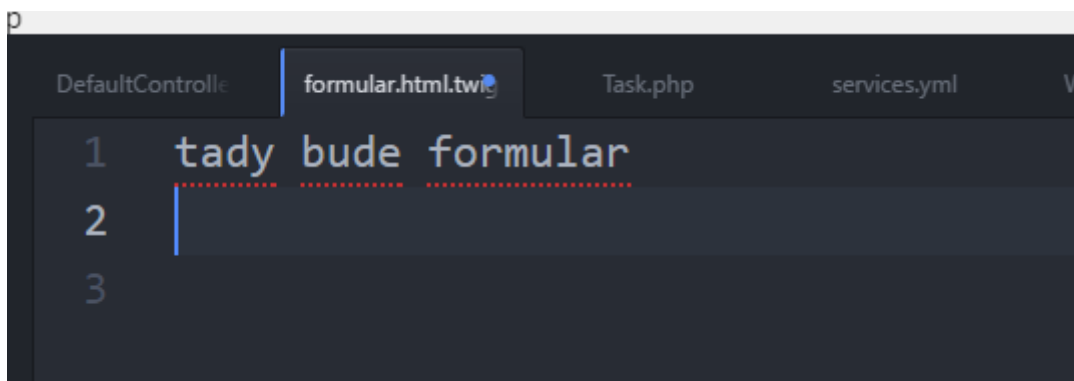
    * @Route("/sluzba" , name="ukazkaservices" )
    */
    public function responzeServicesAction(Request $request)
    {

        $welcomer = $this->get('app.welcomer'); //kde ji vezem, v appconfig servicess.Impl
        $message = $welcomer->create("cizince");
        return new Response($message);
    }
    /**
    * @Route("/formular" , name="formulars" )
    */
    public function formularAction(Request $request)
    {

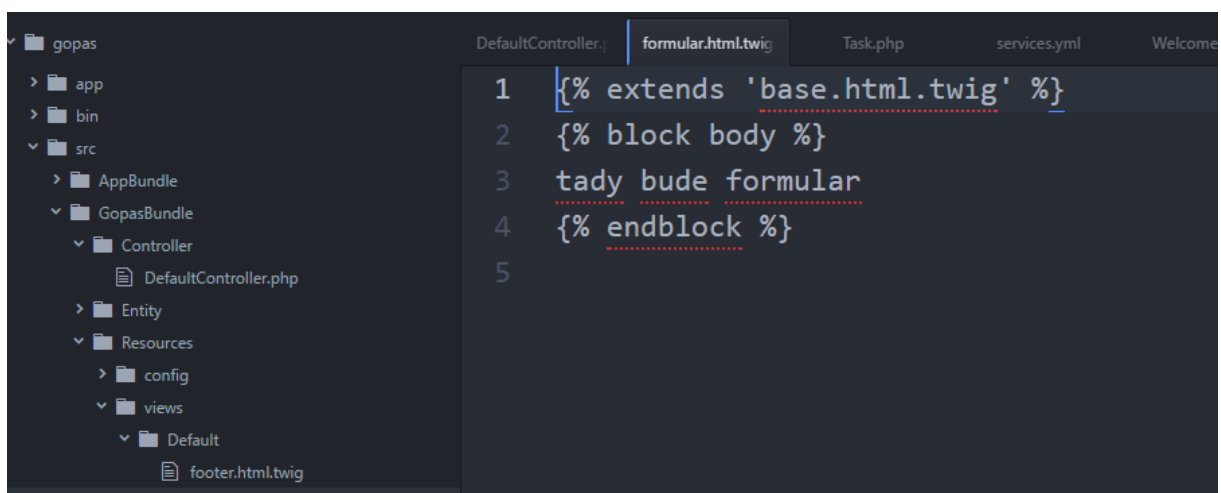
        return $this->render('GopasBundle:Default:formular.html.twig');
    }

```

Vytvořit nový soubor



Upravit





Výše uvedený formulář pracuje.15.04.2017 13:04.

Přidám do use

```
8 use Symfony\Component\HttpFoundation\Response;
9 use Symfony\Component\HttpFoundation\Request;
10 use GopasBundte\Utils\Welcomer;
11 use GopasBundle\utils\Task;
12 class DefaultController extends Controller
13 {
```

Toto vyrobí HTML výstup

```
=>$form->createView(),
```

```
DefaultController.php  .htaccess  formular.html.twig  Task.php  services.yml  Welcomer.php  jmena.html.twig  footer.html.twig  config.cson  index.html.twig

5      $welcomer = $this->get('app.welcomer'); //kde ji vezem, v appconfig services.im
5      $message = $welcomer->create("cizince");
7      return new Response($message);
8  }
9  /**
9  * @Route("/formular" ,name="formulars" )
10 * /
11
12 //tato funkce dela
13 public function formularAction(Request $request)
14 {
15     $task = new Task();
16     //pridat do use
17     $task->setTask("nova uloha");
18     $form = $this->createFormBuilder($task)
19         ->add("task")
20         ->getForm();
21
22     dump($task);
23     return $this->render('GopasBundle:Default:formular.html.twig',
24         array('formular' => $form->createView(),
25     ));
26 }
27
28
29
```

Ještě přidat do controlleru

```
DefaultController.php  Task.php  formular.html.twig

2
3 namespace smrckaBundle\Controller;
4
5 use Symfony\Bundle\FrameworkBundle\Controller\Controller;
6 use Sensio\Bundle\FrameworkExtraBundle\Configuration\Route;
7 use Symfony\Component\HttpFoundation\Response;
8 use Symfony\Component\HttpFoundation\Request;
9 use smrckaBundle\Entity\Task;
10 use Symfony\Component\Form\Extension\Core\Type\TextType;
11 use Symfony\Component\Form\Extension\Core\Type\SubmitType;
12
```

V task.php jsme definovali entitu.

Sestavení formuláře

```
$form = $this->createFormBuilder($task)
    ->add("task")
    ->getForm();
```

```
* @Route("/formular", name="formular")
*/
public function formularAction(Request $request)
{
    $task=new Task();
    $task->setTask("nova uloha");
    $form = $this->createFormBuilder($task)
        ->add("task")

        ->getForm();

    dump($task);
    //return $this->render('smrckaBundle:Default:formular.html.twig');
    return $this->render('smrckaBundle:Default:formular.html.twig',
        array('formular'=>$form->createView(), ));
}
```

Formulář předáme do proměnné formular

	DefaultController	.htaccess	formular.html.twig	Task.php	services.yml
1	{% extends 'base.html.twig' %}				
2	{% block body %}				
3	{% form(formular) %}				
4	{% endblock %}				
5					



Vyzkoušeno 15.4.2017

5 Ve verzi 2024

```
<?php
namespace App\Entity;
class Task{

    protected $task;

    /**
     * @return mixed
     */
    public function getTask()
    {
        return $this->task;
    }

    /**
     * @param mixed $task
     */
    public function setTask($task)
    {
        $this->task = $task;
    }
}
```

controller

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;
use App\Entity\Task;

class FormController extends AbstractController
{
    private $createFormBuilder;
```

```
#[Route('/form', name: 'app_form')]
public function index()
{
    $task= new Task();
    $task->setTask('nova uloha');
    $form=$this->createFormBuilder($task)->add('task')
    ->getForm();

    return $this->render('form/index.html.twig', [
        'formular' => $form->createView()
    ]);
}
```

Poznámka

`$this->createFormBuilder($task)` vytváří nový form builder (stavitel formuláře) s počátečními daty zadanými v argumentu `$task`. Form builder je nástroj, který umožňuje postupné přidávání polí a konfiguraci formuláře, než je formulář samotný vytvořen.

Ve Symfony slouží metoda `$form->createView()` k převodu formuláře na objekt pohledu, který lze následně vykreslit v šabloně. Tato metoda v podstatě připraví formulář na zobrazení, včetně nastavení struktury a atributů potřebných pro správné zobrazení polí formuláře v HTML šabloně. Tento krok je zásadní při práci s formuláři ve Symfony, protože vytváří most mezi PHP objektem formuláře a jeho reprezentací v prohlížeči.

twig

```
{% block body %}
{{ form(formular) }}
{% endblock %}
```

V Symfony slouží konstrukce `{{ form(formular) }}` k vykreslení HTML formuláře v Twig šabloně. Tato šablonová značka vezme objekt formuláře, který byl vytvořen a zpracován ve vašem kontroleru, a vytvoří odpovídající HTML kód pro zobrazení formuláře ve vaší webové stránce.

5.1 Přidáme tlačítko

Přidáme 2 use

```
use Symfony\Component\Form\Extension\Core\Type\TextType;  
use Symfony\Component\Form\Extension\Core\Type\SubmitType;
```

Přidám

```
$form = $this->createFormBuilder($task)  
    ->add("task")  
    ->add('save', SubmitType::class)  
    ->getForm();
```



Ve verzi 6.4

```
$form=$this->createFormBuilder($task)->add('task')  
    ->add('save', SubmitType::class)  
    ->getForm();
```

Na tlačítko přidáme nápis

```
$form = $this->createFormBuilder($task)  
    ->add("task")  
    ->add('save', SubmitType::class, array('label' => 'odeslat ulohu', ))  
    ->getForm();  
  
dump($task);
```

Verze 6.4

```
#[Route('/form', name: 'app_form')]
public function index()
{
    $task= new Task();
    $task->setTask('nova uloha');
    $form=$this->createFormBuilder($task)->add('task')
        ->add('save', SubmitType::class, array('label'=>'Odesli'))
        ->getForm();
}
```



Přidám css

Umí pracovat s třídami bootstrapu.

6 Co se odesílá, tedy zpracování

Po odeslání to jde na tu samou routu

```
DefaultController:  GopasBundle.php:  formular.html.twig  Task.php  services.yml  Welcomer.php  jmena.html.twig  footer.html.twig  c
1  {% extends 'base.html.twig' %}
2  {% block body %}
3  {% for flashMessage in app.session.flashbag.get('notice')%}
4  <div style="color: orange">{{flashMessage}}</div>
5
6  {% endfor %}
7  {{ form(formular) }}
8  {% endblock %}
9
```



```

06     public function formularAction(Request $request)
07     {
08         $task = new Task();
09         //pridat do use
10         $task->setTask("nova uloha");
11
12         $form = $this->createFormBuilder($task)
13             ->add("task", TextType::class, array('label' => 'Nazev
14             ))
15             ->add('save', SubmitType::class, array('label' => 'odesl
16             'attr'=>array('class'=>'btn btn-big')
17             ))
18             ->getForm();
19         // co to vraci
20         $form->handleRequest($request);
21         //zda bzl formular odeslany a validita formulare
22         if ($form->isSubmitted() && $form->isValid()){
23             //zpracovani odeslaneho formulare
24             // v symfony flashbag do nej muzeme poslat hlaseni
25             $this->addFlash('notice', 'Uloha byla odeslana');
26             //flah musime pridat do twigu aby se vykrasli
27
28         }

```

verze 6.4

```

public function index(Request $request)
{
    $task= new Task();
    $task->setTask('nova uloha');
    $form=$this->createFormBuilder($task)->add('task')
        ->add('save', SubmitType::class, array('label'=>'Odesli'))
        ->getForm();
    $form->handleRequest($request);

    return $this->render('form/index.html.twig', [
        'formular' => $form->createView()
    ]);

```

Ve Symfony metoda `$form->handleRequest($request)` zpracovává a předává data z HTTP požadavku (`$request`) do objektu formuláře (`$form`). Tato metoda provádí několik kroků:

1. **Podání dat formuláře:** Pokud je formulář odeslán (například přes POST nebo PUT požadavek), metoda načte data z požadavku.
2. **Validace formuláře:** Data formuláře jsou validována na základě pravidel definovaných v objektu formuláře.

3. **Mapování dat:** Pokud je validace úspěšná, data jsou mapována zpět do objektu modelu nebo entity, která byla použita k vytvoření formuláře.



```
{
    $task = new Task();
    //pridat do use
    $task->setTask("nova uloha");

    $form = $this->createFormBuilder($task)
        ->add("task")
        ->add('save', SubmitType::class, array('label' => 'odeslat ulohu',
        'attr'=>array('class'=>'btn btn-big')
        ))
        ->getForm();
}
```

Změníme název vstupního okna

```

$form = $this->createFormBuilder($task)
    ->add("task", TextType::class, array('label' => 'Nazev ulohy',
    ))
    ->add('save', SubmitType::class, array('label' => 'odeslat ulohu',
    'attr'=>array('class'=>'btn btn-big')
    ))
    ->getForm();

```

Composer x PHP For Windows: Binari x Installing

127.0.0.1:8000/formular

Nejaky nadpis, co je vsude

Nazev ulohy

7 Ve formuláři mohou být 2 tlačítka

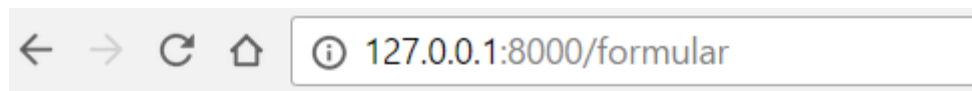
Přidáme tlačítko

```

$form = $this->createFormBuilder($task)
    ->add("task", TextType::class, array('label' => 'Nazev ulohy',
    ))
    ->add('save', SubmitType::class, array('label' => 'odeslat ulohu',
    'attr'=>array('class'=>'btn btn-big')
    ))
    ->add('zpracovat', SubmitType::class, array('label' => 'zpracovat',
    'attr'=>array('class'=>'btn btn-big')
    ))

```

```
DefaultController: GopasBundle.php formular.html.twig Task.php services.yml Welcomer.php jmena.html.twig footer.html.twig config.cson index.htm
127         ->getForm();
128     // co to vraci
129         $form->handleRequest($request);
130     //zda byl formular odeslany a validita formulare
131         if ($form->isSubmitted() && $form->isValid())
132         {
133
134
135             if ($form->get('save')->isClicked()){
136 $this->addFlash('notice','Uloha byla odeslana');
137
138             }
139     //zpracovani odeslaneho formulare
140     // v symfony flashbag do nej muzeme poslat hlasi
141
142     //flash musime pridat do twigu aby se vykрасlil
143     else {
144         $this->addFlash('notice','Uloha byla zpracovana');
145
146     }
147
148 }
```



Nejaky nadpis, co je vsude

Uloha byla zpracovana

Nazev ulohy nova uloha

odeslat ulohu

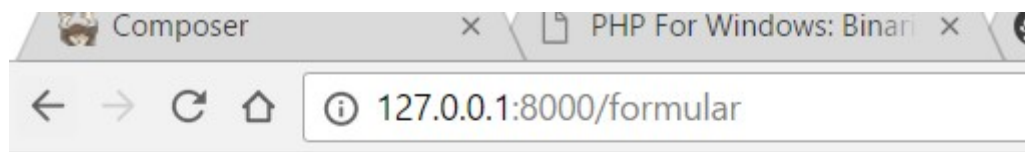
zpracovat

8 Zobrazení odeslaných dat

Přidám getTaskPřidám getTask

```
{

    if ($form->get('save')->isClicked()){
$this->addFlash('notice','Uloha.'| $task->getTask(). 'BYla odeslana');
    }
```



Nejaky nadpis, co je vsude

UlohaaaaaaBYla odeslana

Nazev ulohyaaaa

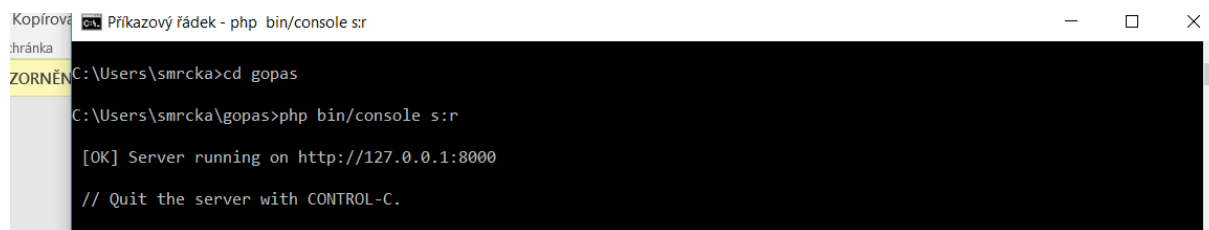
odeslat ulohu

zpracovat

Opět pustím

Přepnu se do smrcka\gopas

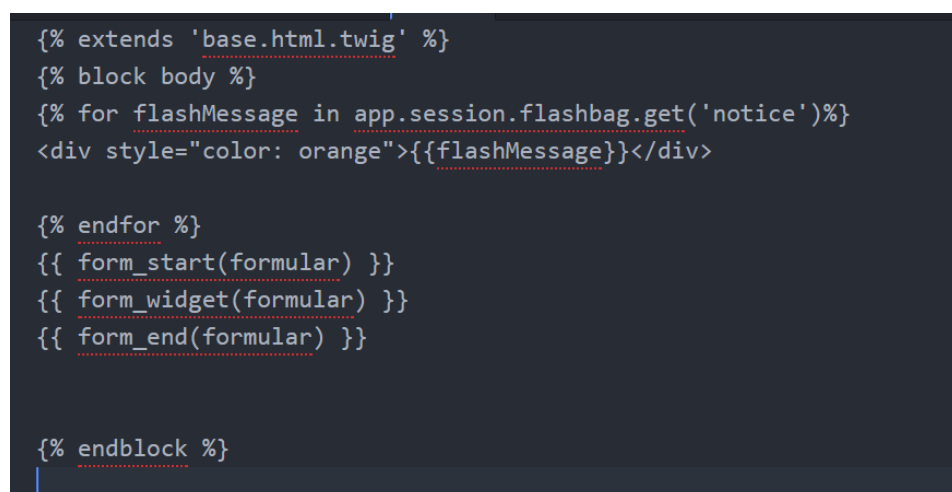
Zde



```
Kopírovat Příkazový řádek - php bin/console sr
C:\Users\smrcka>cd gopas
C:\Users\smrcka\gopas>php bin/console s:r
[OK] Server running on http://127.0.0.1:8000
// Quit the server with CONTROL-C.
```

Jak můžeme zápis rozkouskovat, abychom více ovlivnili html výstup.

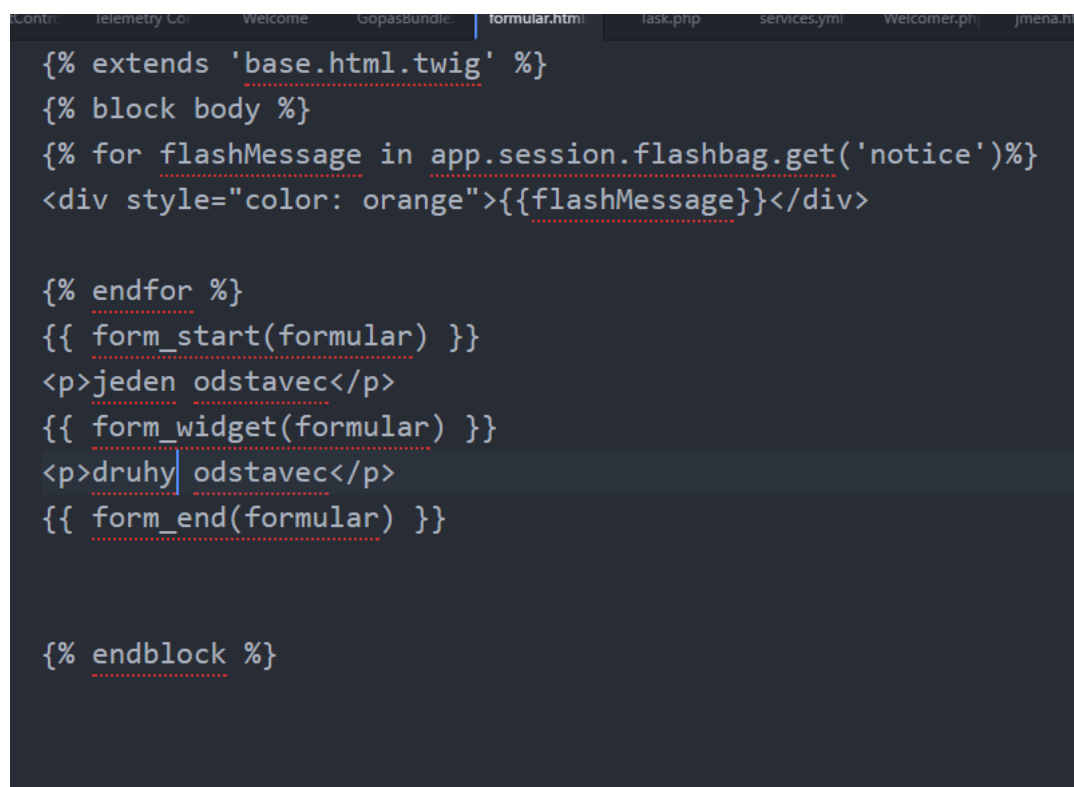
Jeden řádek nahradím:



```
{% extends 'base.html.twig' %}
{% block body %}
{% for flashMessage in app.session.flashbag.get('notice')%}
<div style="color: orange">{{flashMessage}}</div>

{% endfor %}
{{ form_start(formular) }}
{{ form_widget(formular) }}
{{ form_end(formular) }}

{% endblock %}
```



```
{% extends 'base.html.twig' %}
{% block body %}
{% for flashMessage in app.session.flashbag.get('notice')%}
<div style="color: orange">{{flashMessage}}</div>

{% endfor %}
{{ form_start(formular) }}
<p>jeden odstavec</p>
{{ form_widget(formular) }}
<p>druhý odstavec</p>
{{ form_end(formular) }}

{% endblock %}
```

Nejaky nadpis, co je vsude

jeden odstavec

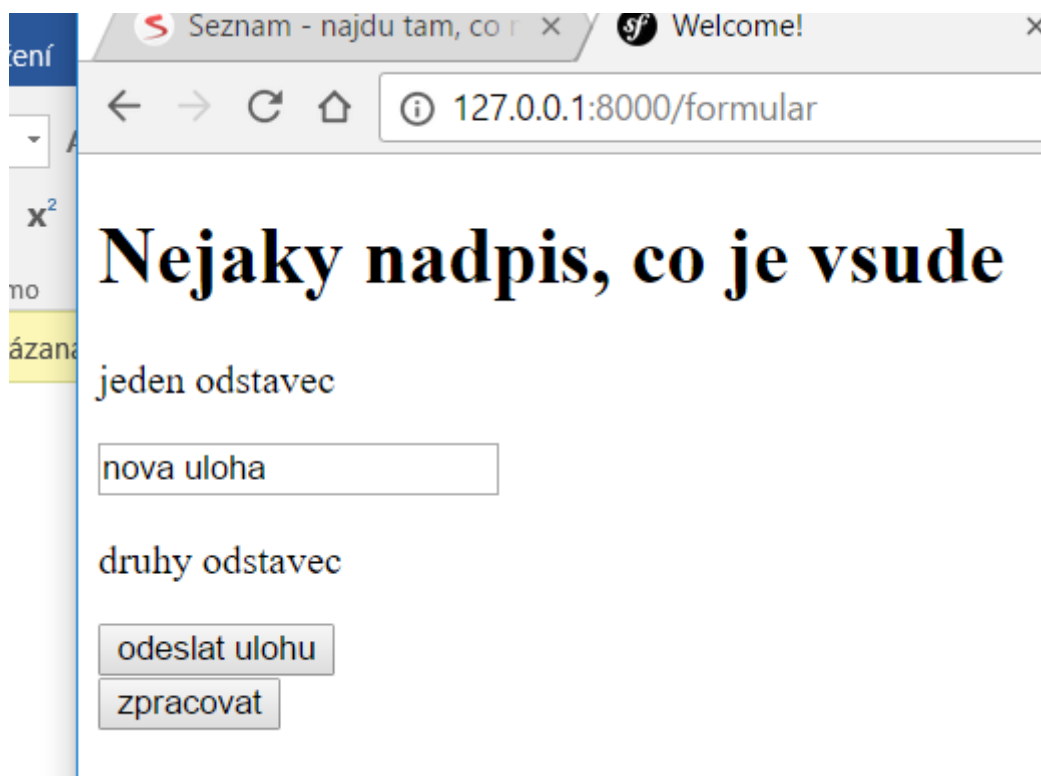
Nazev ulohy

druhy odstavec

Dale přidáme

```
{{ form_widget(formular.task) }}
```

Vykreslí se jen poličko



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/formular'. The page content is identical to the first image, showing the heading 'Nejaky nadpis, co je vsude', the paragraph 'jeden odstavec', the text input field 'nova uloha', the paragraph 'druhy odstavec', and the two buttons 'odeslat ulohu' and 'zpracovat'.

Dále

```
{{ form_widget(formular.task) }}  
{{ form_widget(formular.save) }}
```

pak



Nejaky nadpis, co je vsude

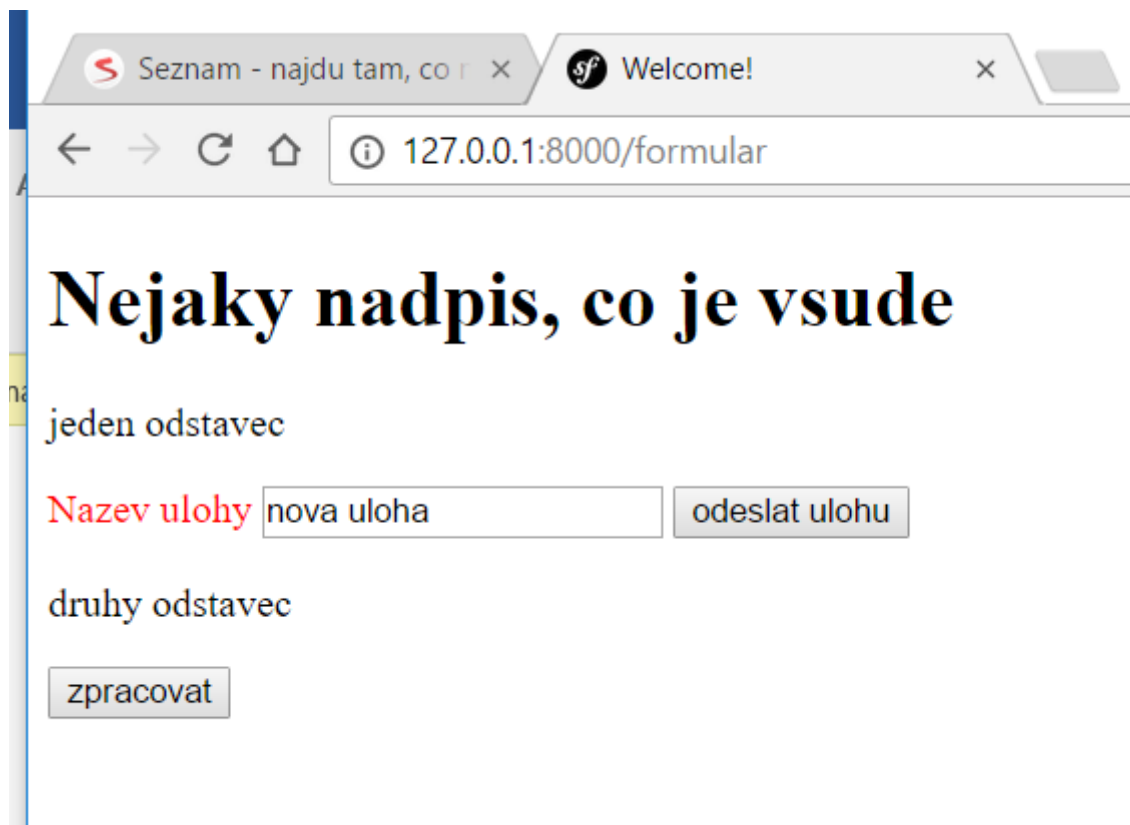
jeden odstavec

nova uloha

druhy odstavec

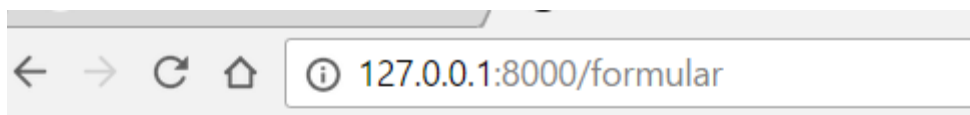
Dále

```
<p>jeden odstavec</p>  
<span style='color:red'>{{ form_label(formular.task) }}</span>
```



Dále

```
<span style='color:red'>{{ form_label(formular.task,"Uloha") }}</span>
```



Nejaky nadpis, co je vsude

jeden odstavec

Uloha

druhy odstavec

Tedy label nastavíme buď v controlleru nebo z šablony voláním label.

9 Zobrazení chyb ve formuláři

Formo errors

Vypíše všechny chyby.

```
{% endfor %}
{{ form_start(formular) }}
{{ form_errors(formular) }}

<p>jeden odstavec</p>
<span style='color:red'>{{ form_label(formul
```

Jen chyby políčka.

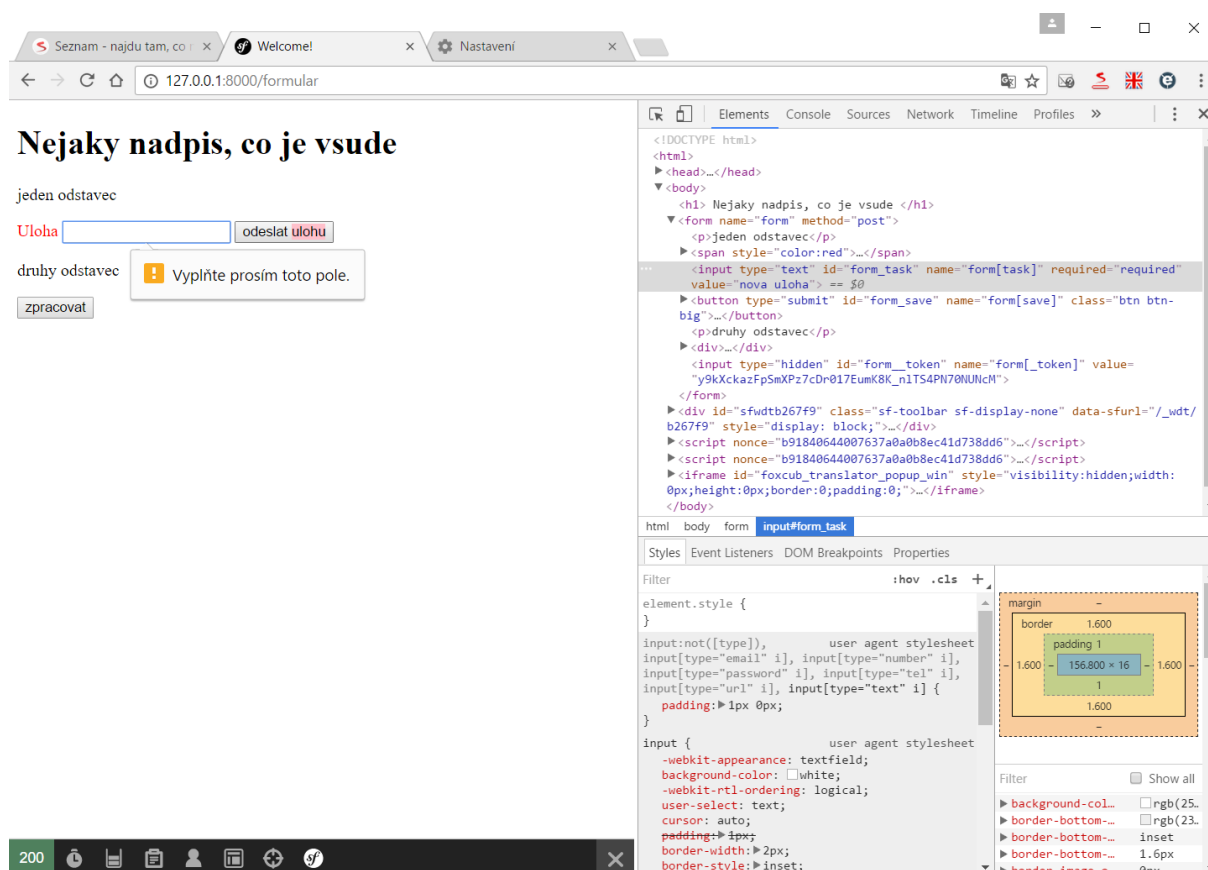
```
{{ form_widget(formular.task) }}
{{ form_errors(formular.task) }}
{{ form_widget(formular.save) }}
<p>druhy odstavec</p>
{{ form_end(formular) }}
```

Kontrola na povinné zadání.

```
//pridat do use
$task->setTask("nova uloha");

$form = $this->createFormBuilder($task)
    ->add("task", TextType::class, array('label' => 'Nazev ulohy',
        'required'=>true,
    ))
    ->add('save', SubmitType::class, array('label' => 'odeslat ulohu',

        'attr'=>array('class'=>'btn btn-big')
    ))
```



10 Zabezpečení zadávání ve formuláři na úrovni serveru

v Task.php

Pomocí php anotace

```
namespace GopasBundle\Entity;
use Symfony\Component\Validator\Constraints as Assert;
class Task{
    //promenna task

    ->add("task", TextType::class, array('label' => 'Nazev ulohy',
        'required'=>false, // html5 validace povinnosti vyplnit
    ))
    ->add('save', SubmitType::class, array('label' => 'odeslat ulohu'
```

<.php

```
namespace GopasBundle\Entity;
use Symfony\Component\Validator\Constraints as Assert;
class Task{
    //promenna task
    //vsechny protected, vracet gettr nap pro task getTask
    // opak gettr je _set_oid_output_format

    //mame entitu s jednou vlastnosti
    //zacatek php anotaci, chci aby policko nebylo prazdne
    /**
     * @Assert\NotBlank()
     * @Assert\Length(
     *     min = 2,
     *     max = 2,
     *     minMessage = "zadejte delsi ulohu",
     *     maxMessage = "zadejte kratši ulohu",
     *)
    */
    protected $task;

    public function getTask(){

        return $this->task;
    }
}
```

Nejaky nadpis, co je vsude

jeden odstavec

Uloha

- zadejte kratši ulohu

odeslat ulohu

druhy odstavec

zpracovat

```
{% extends 'base.html.twig' %}
{% block body %}
{% for flashMessage in app.session.flashbag.get('notice')%}
<div style="color: orange">{{ flashMessage }}</div>

{% endfor %}
{{ form_start(formular) }}
{{ form_errors(formular) }}

<p>jeden odstavec</p>
<span style='color:red'>{{ form_label(formular.task, "Uloha") }}</span>

{{ form_widget(formular.task) }}
{{ form_errors(formular.task) }}
{{ form_widget(formular.save) }}
<p>druhy odstavec</p>
{{ form_end(formular) }}

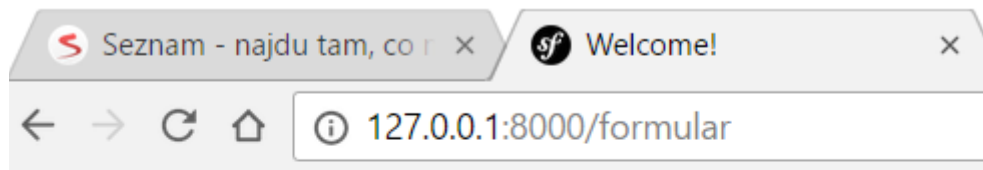
{% endblock %}
```

11 Dokumentace Symfony validation

<http://symfony.com/doc/current/validation.html>

například doplnit

```
4  class Task{
5      //promenna task
6      //vsechny protected, vracet gettr nap pro task getTask
7      // opak gettr je _set_oid_output_format
8
9      //mame entitu s jednou vlastnosti
10     //zacatek php anotaci, chci aby policko nebylo prazdne
11     /**
12      * @Assert\NotBlank(
13      *     message = "zadejte prisim nazev ulohy"
14      * )
15      * @Assert\Length(
16      *     min = 2,
17      *     max = 5,
18      *     minMessage = "zadejte delsi ulohu",
19      *     maxMessage = "zadejte kratši ulohu",
20      * )
21     */
```



Nejaky nadpis, co je vsude

jeden odstavec

Uloha

- zadejte prisim nazev ulohy

odeslat ulohu

druhy odstavec

zpracovat

12 Callback – lze definovat vlastní funkci

Callback

The purpose of the Callback constraint is to create completely custom validation rules and to assign any validation errors to specific fields on your object. If you're using validation with forms, this means that you can make these custom errors display next to a specific field, instead of simply at the top of your form.

3.2 version

[edit this page](#)

This process works by specifying one or more *callback* methods, each of which will be called during the validation process. Each of those methods can do anything, including creating and assigning validation errors.



A callback method itself doesn't *fail* or return any value. Instead, as you'll see in the example, a callback method has the ability to directly add validator "violations".

Applies to	class
Options	• callback • payload
Class	Callback
Validator	CallbackValidator

13 Twig ovládání dalších vlastností funkce set

Použití dump

```
{{ form_widget(formular.task) }}
{{ form_errors(formular.task) }}
{{ form_widget(formular.save) }}
<p>druhy odstavec</p>
{{ form_end(formular) }}
{% dump(formular.task.vars) %}
```

Vydumpovalo to



http://127.0.0.1:8000/formular

Method: POST HTTP Status: 200 IP: 127.0.0.1 Profiled on:
17 Mar 2017 08:54:26 +0000 Token: 72f5a2

Dumped Contents

In DefaultController.php line 147:

```
Task {#40 ▼  
  #task: null  
}
```

In GopasBundle:Default:formular.html.twig line 22:

```
array:24 [▼  
  "value" => ""  
  "attr" => []  
  "form" => FormView {#402 ▶}  
  "id" => "form_task"  
  "name" => "task"  
  "full_name" => "form[task]"  
  "disabled" => false  
  "label" => "Nazev ulohy"  
  "label_format" => null  
  "multipart" => false  
  "block_prefixes" => array:3 [▶]  
  "unique_block_prefix" => "_form_task"  
  "translation_domain" => null  
  "cache_key" => "_form_task_text"  
  "errors" => FormErrorIterator {#443 ▶}  
  "valid" => false  
  "data" => null  
  "required" => false  
  "size" => null  
  "label_attr" => []  
  "compound" => false
```

Ve twigu je možné dat action akci - vytvořit v controlleru.